

UNIVERSITÀ TELEMATICA PEGASO

Corso di Laurea Triennale in Informatica per le aziende digitali — L-31

RELAZIONE TECNICA BACKEND DATABASE

Ingegnerizzazione del backend dati: modello relazionale, migrazioni, sicurezza e diagnostica

Project Work n. 19 — Tema 2: Privacy e sicurezza aziendale

Progettazione e realizzazione di una base dati relazionale per asset, servizi, dipendenze e responsabilità in ambito NIS2/ACN

Autore: Gianluca Lucarelli

Versione 2.1 — Luglio 2026

Allegato tecnico di approfondimento al documento principale del Project Work

Nota sul documento

La presente relazione costituisce l'allegato tecnico dedicato al backend del Project Work. Il documento principale del PW rimane l'elaborato di riferimento per la presentazione complessiva del progetto; questa relazione ne approfondisce invece le scelte di modellazione, la pipeline SQL, la sicurezza, la tracciabilità e le verifiche eseguite sul database PostgreSQL gestito tramite Supabase.

La versione 2.1 documenta lo stato consolidato del backend, aggiorna i due schemi interni e recepisce l'evoluzione effettiva del sistema: la pipeline produttiva comprende gli script 01–21, il toolkit diagnostico gli script X1–X17 e il modello finale è composto da 30 tabelle. Le parti storiche relative alle prime versioni dell'audit e del reporting sono mantenute come evidenza del refactoring, ma sono chiaramente distinte dallo stato operativo finale.

Criterio di affidabilità: i conteggi riportati come stato finale sono quelli verificati dagli script produttivi 20–21 e dalla diagnostica X17. I conteggi complessivi di oggetti interni alla piattaforma Supabase, non direttamente attribuibili al progetto, non vengono utilizzati come evidenza tecnica principale.

Indice dei contenuti

Selezionare una voce dell'indice per raggiungere direttamente la relativa sezione.

- [1. Finalità e perimetro della relazione](#)
- [2. Architettura del backend e organizzazione del repository](#)
- [3. Stato finale verificato del database](#)
- [4. Core Migration Pipeline \(01–21\)](#)
 - [4.1 01-Inizializzazione-Schema-v6.8_Hardened.sql](#)
 - [4.2 02-dati.sql](#)
 - [4.3 03-sicurezza-rls.sql](#)
 - [4.4 04-Fix schema permissions for Supabase.sql](#)
 - [4.5 05 viste_esportazione_acn.sql](#)
 - [4.6 06 configurazione_audit_permissions.sql](#)
 - [4.7 07 impostazione_timezone_locale.sql](#)
 - [4.8 08-update-read-policy.sql](#)
 - [4.9 09-tassonomia-incidenti-acn.sql](#)
 - [4.10 10-popolamento-tassonomia-acn.sql](#)
 - [4.11 11-policy-evento-servizio.sql](#)
 - [4.12 12-Hardening-Vincoli-Dominio-Organizzazione-v1.0.sql](#)
 - [4.13 13-Hardening-Gerarchia-Servizi-v1.0.sql](#)
 - [4.14 14-Hardening-Gerarchia-Asset-v1.0.sql](#)
 - [4.15 15-Hardening-Gerarchia-Fornitori-v1.0.sql](#)
 - [4.16 16-Hardening-Relazione-Asset-Fornitore-v1.0.sql](#)
 - [4.17 17-Completamento-Supply-Chain-Multilivello-v1.0.sql](#)
 - [4.18 18-Correzione-Vista-Reporting-Servizi-Critici-v1.0.sql](#)
 - [4.19 19-Estensione-Sistema-Audit-Generalizzato-v1.0.sql](#)
 - [4.20 20-Uniformazione-Archiviazione-Logica-v1.0.sql](#)
 - [4.21 21-Correzione-Vincolo-Responsabile-Ruolo-v1.0.sql](#)
- [5. Diagnostic, Validation & Patch Toolkit \(X1–X17\)](#)

5.1	<u>X1_EXPORT_SCHEMA_db.sql</u>
5.2	<u>X2_VISUALIZZA_COLONNE_VISTE.sql</u>
5.3	<u>X3_SETUP_SUPPLY_CHAIN.sql</u>
5.4	<u>X4_FIX_SUPPLY_CHAIN_FW.sql</u>
5.5	<u>X5_AUDIT_TRAIL_SETUP.sql</u>
5.6	<u>X6_EXPORT_TRIGGER_FUNCTIONS.sql</u>
5.7	<u>X7_EXPORT_DATABASE_CONSTRAINTS.sql</u>
5.8	<u>X8_CHECK_DUPLICATES_AND_NULL_ORGANIZATIONS.sql</u>
5.9	<u>X9_EXPORT_VIEWS_DEFINITIONS_AND_PRIVILEGES.sql</u>
5.10	<u>X10_EXPORT_RLS_POLICIES_AND_GRANTS.sql</u>
5.11	<u>X11–X12: readiness del modello fornitori</u>
5.12	<u>X13_CHECK_SUPPLY_CHAIN_MULTILIVELLO.sql</u>
5.13	<u>X14: diagnostica della vista di reporting</u>
5.14	<u>X15_CHECK_AUDIT_SYSTEM_READINESS.sql</u>
5.15	<u>X16_CHECK_ARCHIVIAZIONE_LOGICA_READINESS.sql</u>
5.16	<u>X17_CHECK_VERIFICA_FINALE_BACKEND.sql</u>
6.	<u>Analisi delle ridondanze e refactoring controllato</u>
6.1	<u>Evoluzione dell’Audit Engine</u>
6.2	<u>Abilitazione RLS reiterata</u>
6.3	<u>Riallineamento delle policy di lettura</u>
6.4	<u>Correzione della vista di reporting</u>
6.5	<u>Dalla cancellazione fisica alla conservazione logica</u>
7.	<u>Modello relazionale, gerarchie e normalizzazione</u>
7.1	<u>Gerarchie storicizzabili</u>
7.2	<u>Integrità referenziale e domini</u>
8.	<u>Supply Chain multilivello</u>
9.	<u>Sicurezza: RLS, MFA, privilegi e viste</u>
9.1	<u>Lettura e scrittura</u>
9.2	<u>Ruolo anon</u>
9.3	<u>Cancellazione</u>
9.4	<u>FORCE ROW LEVEL SECURITY</u>
10.	<u>Audit e tracciabilità</u>
11.	<u>Archiviazione logica e conservazione storica</u>
12.	<u>Reporting e interoperabilità ACN/NIS2</u>
13.	<u>Verifica finale del backend</u>
14.	<u>Riproducibilità, repository e gestione delle versioni</u>
15.	<u>Conclusioni</u>
	<u>Appendice A — Matrice di controllo documentale e applicativa</u>

1. Finalità e perimetro della relazione

Il progetto realizza un registro centralizzato per la catalogazione degli asset tecnologici, dei servizi critici, delle dipendenze, dei fornitori e delle responsabilità organizzative utili alla costruzione dei profili richiesti dall'Agenzia per la Cybersicurezza Nazionale nel quadro della direttiva NIS2. Il backend è stato progettato come una base dati relazionale normalizzata, sottoposta a vincoli di integrità, politiche Row Level Security e controlli di tracciabilità. Il modello è trasversale e applicabile a organizzazioni pubbliche e private; gli eventuali dati simulati non ne circoscrivono l'impiego a uno specifico settore.

La relazione documenta il percorso che ha portato dalla baseline iniziale al modello consolidato. Il perimetro comprende lo schema public di PostgreSQL, gli script SQL versionati, le viste applicative, le funzioni e i trigger principali, il sistema di audit, l'archiviazione logica e gli strumenti diagnostici usati per validare ogni evoluzione. Non rientrano invece nel dettaglio di questo allegato l'implementazione delle schermate web e i flussi SCR, che saranno descritti nel documento docs/Relazione_Tecnica_Frontend_Applicativo.pdf.

L'obiettivo non è soltanto descrivere gli oggetti presenti nel database, ma dimostrare che il loro sviluppo è avvenuto attraverso una sequenza riproducibile, verificabile e coerente con i requisiti di sicurezza del Project Work.

2. Architettura del backend e organizzazione del repository

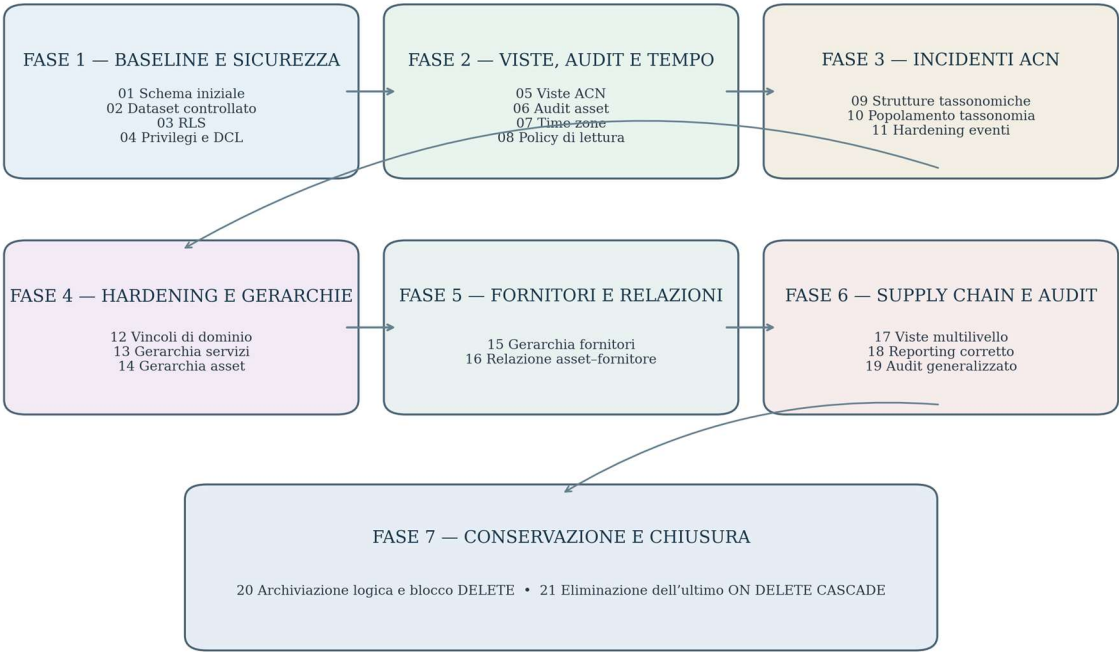
Il database è implementato su PostgreSQL attraverso un'istanza cloud Supabase. Lo schema applicativo public contiene le tabelle, le viste, le funzioni e i trigger sviluppati per il progetto. I ruoli applicativi anon e authenticated sono gestiti separatamente dalle policy Row Level Security: i privilegi PostgreSQL definiscono quali operazioni sono astrattamente disponibili, mentre le policy RLS stabiliscono quali righe possono essere lette o modificate in relazione al contesto di autenticazione.

La gestione del codice SQL si basa su due rami logici complementari: la Core Migration Pipeline, formata dagli script produttivi numerati 01–21, e il Diagnostic, Validation & Patch Toolkit, formato dagli script X1–X17. La prima modifica lo stato del database in modo sequenziale e controllato; il secondo ispeziona metadati, dati e configurazioni, producendo evidenze tecniche prima e dopo le migrazioni.

Il flusso operativo adottato nel progetto prevede l'esecuzione iniziale dello script su Supabase, la verifica dell'esito, il salvataggio nel repository mediante `vscode.dev/GitHub` e, infine, la creazione della copia locale su Google Drive. Questa procedura evita di versionare file non ancora verificati e mantiene allineati ambiente cloud, repository remoto e archivio locale.

CORE MIGRATION PIPELINE (01-21)

Evoluzione sequenziale del backend PostgreSQL/Supabase



Risultato verificato: 30 tabelle, 7 viste principali, sicurezza RLS, audit generalizzato e conservazione logica.

Figura 1 — Core Migration Pipeline aggiornata: dalla baseline 01 alla chiusura del backend con lo script 21.

3. Stato finale verificato del database

La diagnostica finale X17, rieseguita dopo la correzione dell'ultimo vincolo ON DELETE CASCADE, ha restituito l'esito VERIFICA_FINALE_BACKEND_SUPERATA. Il risultato consente di distinguere i conteggi effettivamente certificati dalle semplici osservazioni dell'interfaccia Supabase.

Indicatore	Valore	Significato
Tabelle nello schema public	30	Conteggio verificato da X17
Tabelle operative principali	14	RLS, audit e blocco DELETE verificati
Viste applicative principali	7	Tutte presenti, security_invoker e non accessibili ad anon
Trigger di audit	14	Uno per ciascuna tabella operativa coperta
Trigger di blocco DELETE	14	Cancellazione fisica impedita sulle tabelle operative
Trigger di archiviazione standard	10	Gestione automatica dei metadati di archiviazione

Indicatore	Valore	Significato
Vincoli ON DELETE CASCADE	0	Eliminati dallo schema public con gli script 20–21
Policy DELETE	0	Nessuna policy applicativa di cancellazione
Privilegi anon	0	Nessun privilegio su tabelle e viste applicative
Indici non validi	0	Verifica dei cataloghi PostgreSQL
Vincoli non validati	0	Tutti i vincoli risultano validati
Duplicati nella vista di reporting	0	Una riga per servizio critico

L'istanza Supabase contiene anche oggetti interni e di piattaforma, quali ruoli di servizio, estensioni e funzioni non sviluppate specificamente per il Project Work. Per questa ragione la relazione attribuisce valore probatorio soltanto agli oggetti applicativi censiti e alle metriche esplicitamente controllate dagli script diagnostici.

4. Core Migration Pipeline (01–21)

La sequenza numerata garantisce una ricostruzione ordinata del backend e documenta l'evoluzione del modello. Le prime quattordici migrazioni costituiscono il nucleo originario del backend e sono descritte nei paragrafi seguenti, insieme alle estensioni 15–21 che completano il modello dei fornitori, la Supply Chain multilivello, l'audit generalizzato, il reporting e la conservazione logica.

4.1 01-Inizializzazione-Schema-v6.8_Hardened.sql

DDL e schema relazionale core

Costituisce il punto di ingresso dell'architettura. Lo script crea le 22 tabelle della baseline relazionale, definisce chiavi primarie, chiavi esterne, vincoli CHECK e le prime strutture di audit. La versione iniziale dell'audit log è centrata sulla tabella asset e serializza gli stati OLD e NEW in forma testuale. Questa impostazione viene mantenuta nella documentazione come fase storica e successivamente superata dall'audit generalizzato dello script 19.

4.2 02-dati.sql

Popolamento dei dizionari e dataset simulato

Gestisce il seed controllato delle tabelle iniziali. La procedura rispetta l'ordine imposto dalle chiavi esterne e popola organizzazioni, domini, asset, servizi, fornitori, dipendenze e dati di test. L'uso di TRUNCATE ... RESTART IDENTITY CASCADE appartiene alla fase di inizializzazione controllata; nello stato finale del sistema la cancellazione fisica delle entità operative è invece impedita dallo script 20.

4.3 03-sicurezza-rls.sql

Automazione delle policy Row Level Security

Abilita la RLS e ricrea le policy applicative secondo il modello uniforme del progetto. La lettura è consentita alle sessioni MFA con livello aal2 oppure all'utenza docente autorizzata docentepegaso@gmail.com. Inserimento e aggiornamento sono consentiti soltanto sulle tabelle operative e richiedono aal2. Non viene definita alcuna policy DELETE.

4.4 04-Fix schema permissions for Supabase.sql

DCL e modello Zero Trust

Configura i privilegi PostgreSQL separandoli dalle condizioni RLS. Il ruolo anon non riceve privilegi sulle tabelle e sulle sequenze applicative. Il ruolo authenticated ottiene SELECT sulle risorse autorizzate e INSERT/UPDATE soltanto sulle tabelle operative. Il privilegio DELETE non viene concesso. Lo script imposta inoltre privilegi predefiniti prudenti per gli oggetti creati in seguito.

4.5 05_viste_esportazione_acn.sql

Reporting e interoperabilità ACN/NIS2

Introduce vista_esportazione_acn_assets e vista_reporting_servizi_critici, entrambe configurate con security_invoker. La prima aggrega asset, categorie ACN e vulnerabilità. La seconda collega servizi, asset e fornitori. La prima versione della vista di reporting presentava una moltiplicazione delle righe in presenza di più asset e più fornitori; l'anomalia è stata diagnosticata da X14 e risolta definitivamente dallo script 18.

4.6 06_configurazione_audit_permissions.sql

Primo hardening dell'Audit Engine

Consolida l'audit iniziale sulla tabella asset mediante una funzione SECURITY DEFINER e un trigger DML. La tabella audit_log rimane leggibile dagli utenti autorizzati ma non scrivibile direttamente dal frontend. Questa fase introduce il principio di separazione tra operazioni applicative e scrittura del registro, poi esteso a quattordici tabelle dallo script 19.

4.7 07_impostazione_timezone_locale.sql

Allineamento temporale di sistema

Imposta Europe/Rome come fuso orario predefinito delle nuove sessioni PostgreSQL. La scelta garantisce coerenza con il contesto italiano e riduce le ambiguità nella visualizzazione di audit, eventi e dati temporali, mantenendo PostgreSQL responsabile della corretta gestione delle transizioni CET/CEST.

4.8 08-update-read-policy.sql

Riallineamento della policy di lettura

Rimuove eventuali varianti intermedie troppo permissive e ricrea Read_All_Policy secondo il modello MFA/docente. La lettura non è concessa genericamente a tutti gli utenti autenticati, ma soltanto alle sessioni aal2 o all'utenza docente autorizzata. Le policy di scrittura restano separate.

4.9 09-tassonomia-incidenti-acn.sql

Integrazione della Tassonomia Cyber ACN

Estende lo schema con tassonomia_incidenti_acn ed evento_tassonomia_acn. La prima tabella conserva codici, macro-aree, sottocategorie e descrizioni; la seconda realizza l'associazione

multi-a-molti tra eventi e classificazioni. Il vincolo sul passo del wizard mantiene il processo applicativo entro i valori previsti.

4.10 10-popolamento-tassonomia-acn.sql

Popolamento controllato della tassonomia

Inserisce le voci tassonomiche necessarie all'applicazione utilizzando ON CONFLICT (codice_acn) DO NOTHING. La separazione tra definizione DDL e popolamento DML rende la pipeline più chiara e consente di rieseguire il seed senza duplicare i codici ACN.

4.11 11-policy-evento-servizio.sql

Hardening RLS degli eventi di servizio

Elimina le policy legacy permissive e applica a evento_servizio le policy canoniche Read_All_Policy, Insert_MFA_Policy e Update_MFA_Policy. Concede a authenticated soltanto SELECT, INSERT e UPDATE e conferma l'assenza di DELETE. Lo script porta la gestione degli incidenti allo stesso livello di sicurezza delle altre tabelle operative.

4.12 12-Hardening-Vincoli-Dominio-Organizzazione-v1.0.sql

Vincoli di dominio e responsabilità organizzativa

Bonifica i servizi privi di organizzazione e rende obbligatorie le associazioni organizzative di asset e servizi. Introduce inoltre vincoli UNIQUE sui principali dizionari applicativi. La migrazione opera in transazione e si interrompe in presenza di condizioni incoerenti, evitando stati intermedi.

4.13 13-Hardening-Gerarchia-Servizi-v1.0.sql

Gerarchia servizio-sottoservizio

Introduce un codice applicativo stabile per i servizi e trasforma servizio_componente in una relazione gerarchica storicizzabile. Vincoli, indici parziali e trigger impediscono autorelazioni, duplicati attivi, padri primari multipli, cicli e riattivazioni di relazioni già chiuse.

4.14 14-Hardening-Gerarchia-Asset-v1.0.sql

Gerarchia asset-sotto-asset

Estende asset con codice_asset, rende obbligatoria la categoria e crea tipo_relazione_asset e asset_componente. La relazione conserva stato, periodo di validità, padre primario e ordine di visualizzazione. I controlli bloccano autorelazioni, duplicati, cicli, padri primari multipli e riattivazioni storiche. Il modello raggiunge 26 tabelle.

4.15 15-Hardening-Gerarchia-Fornitori-v1.0.sql

Gerarchia fornitore-subfornitore

Aggiunge codice_fornitore come identificativo stabile, completa la classificazione delle tipologie di fornitore e crea tipo_relazione_fornitore e fornitore_relazione. La relazione è storicizzabile, ammette più legami gerarchici ma un solo padre primario attivo e impedisce cicli, autorelazioni e riattivazioni di record storici. Il modello passa a 28 tabelle.

4.16 16-Hardening-Relazione-Asset-Fornitore-v1.0.sql

Relazione esplicita asset-fornitore

Crea tipo_relazione_asset_fornitore e asset_fornitore. La nuova struttura permette di distinguere fornitura, manutenzione, supporto, hosting e altre tipologie di dipendenza. Conserva validità temporale, relazione primaria e stato attivo, impedendo duplicati e più fornitori primari attivi per lo stesso asset e la stessa tipologia. Il modello raggiunge 30 tabelle.

4.17 17-Completamento-Supply-Chain-Multilivello-v1.0.sql

Viste gerarchiche e Supply Chain multilivello

Crea quattro viste security_invoker: vista_gerarchia_servizi_espanza, vista_gerarchia_asset_espanza, vista_gerarchia_fornitori_espanza e vista_supply_chain_multilivello. Quest'ultima distingue le relazioni dirette servizio-fornitore da quelle derivate attraverso servizio-asset-fornitore, rendendo leggibile la catena di dipendenze senza duplicare la struttura fisica delle tabelle.

4.18 18-Correzione-Vista-Reporting-Servizi-Critici-v1.0.sql

Correzione del reporting dei servizi critici

Sostituisce la join moltiplicativa della vista originaria con aggregazioni separate di asset e fornitori. La vista conserva le colonne compatibili con l'interfaccia precedente e aggiunge identificativi, conteggi e strutture JSON. Il risultato garantisce una sola riga per servizio, come confermato dalla diagnostica finale.

4.19 19-Estensione-Sistema-Audit-Generalizzato-v1.0.sql

Audit generalizzato su entità e relazioni operative

Estende audit_log con campi dedicati a tabella, tipo di entità, record, codici e contesti di asset, servizio e fornitore. Registra UUID, e-mail, AAL, ruolo JWT, ruolo database e origine dell'operazione, oltre ai payload JSONB precedente e successivo. Introduce fn_audit_generico, quattordici trigger e vista_audit_dettagliato. Gli eventi legacy vengono preservati e normalizzati.

4.20 20-Uniformazione-Archiviazione-Logica-v1.0.sql

Archiviazione logica e blocco delle cancellazioni fisiche

Aggiunge attiva e i metadati archiviato_il, archiviato_da e motivo_archiviazione alle dieci tabelle prive di storicizzazione. Mantiene valido_dal e valido_al nelle quattro relazioni temporali. Crea dieci trigger di gestione dei metadati e quattordici trigger BEFORE DELETE che impediscono la cancellazione fisica. Sostituisce inoltre i vincoli CASCADE individuati da X16 con RESTRICT.

4.21 21-Correzione-Vincolo-Responsabile-Ruolo-v1.0.sql

Eliminazione dell'ultimo ON DELETE CASCADE

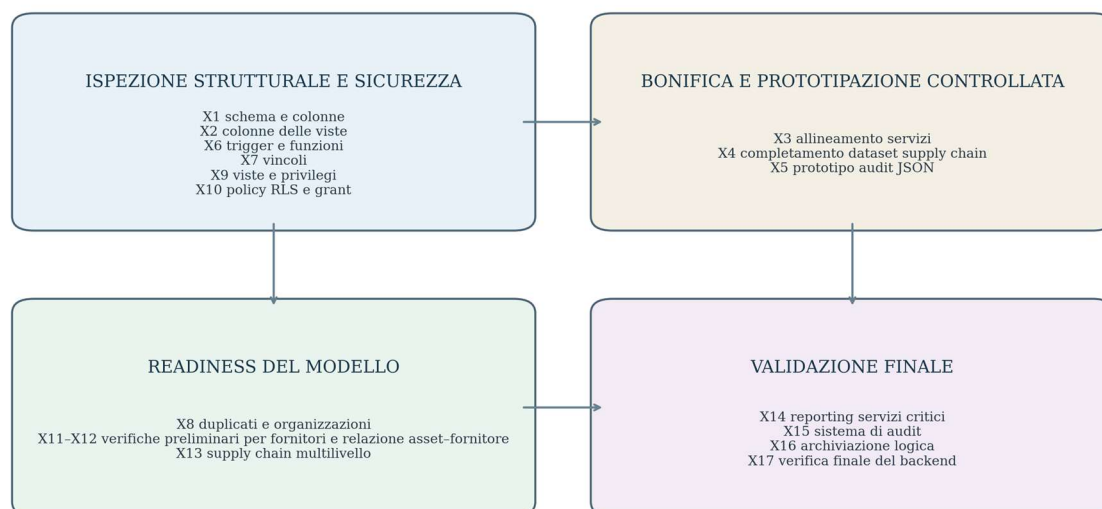
La verifica X17 ha individuato un ultimo vincolo CASCADE tra responsabile_ruolo e ruolo, non compreso nel filtro operativo dello script 20. La migrazione 21 lo sostituisce con RESTRICT e conferma l'assenza totale di chiavi esterne ON DELETE CASCADE nello schema public. Questa correzione consente a X17 di concludersi con esito positivo.

5. Diagnostic, Validation & Patch Toolkit (X1–X17)

La serie X raccoglie gli script impiegati per osservare il database live, verificare la readiness delle migrazioni, individuare anomalie e produrre evidenze di chiusura. Le prime versioni includono anche patch e prototipi controllati; gli strumenti più recenti sono prevalentemente diagnostici di sola lettura.

DIAGNOSTIC, VALIDATION & PATCH TOOLKIT (X1-X17)

Controlli progressivi sul database live e verifiche di readiness



Esito conclusivo di X17: VERIFICA_FINALE_BACKEND_SUPERATA

Il toolkit documenta sia l'evoluzione progettuale sia le evidenze tecniche usate per autorizzare le migrazioni produttive.

Figura 2 — Toolkit aggiornato: dalle ispezioni strutturali alla verifica finale X17.

5.1 X1_EXPORT_SCHEMA_db.sql

Estrae tabelle, colonne, tipi e nullabilità dallo schema public. È la base per il Data Dictionary e permette di verificare lo schema fisico senza dipendere dall'interfaccia grafica.

5.2 X2_VISUALIZZA_COLONNE_VISTE.sql

Analizza le colonne e i tipi esposti dalle viste applicative, supportando la verifica dell'interoperabilità con il frontend e con gli esportatori.

5.3 X3_SETUP_SUPPLY_CHAIN.sql

Esegue una bonifica controllata dei servizi del dataset simulato e dimostra la necessità di query idempotenti. La presenza di LIMIT 1 appartiene alla fase di hardening precedente ai vincoli UNIQUE introdotti successivamente.

5.4 X4_FIX_SUPPLY_CHAIN_FW.sql

Completa il dataset di prova inserendo un fornitore e una dipendenza servizio-fornitore, rendendo verificabile la catena di fornitura nel caso d'uso organizzativo simulato, senza vincolare il modello a uno specifico settore pubblico o privato.

5.5 X5_AUDIT_TRAIL_SETUP.sql

Documenta il prototipo JSON dell'audit, basato su to_jsonb(OLD) e to_jsonb(NEW). Il prototipo non rappresenta lo stato finale, ma costituisce un passaggio progettuale verso lo script 19.

5.6 X6_EXPORT_TRIGGER_FUNCTIONS.sql

Esporta trigger e funzioni non interne e consente di distinguere gli oggetti del progetto da quelli appartenenti agli schemi di servizio di Supabase.

5.7 X7_EXPORT_DATABASE_CONSTRAINTS.sql

Estrae chiavi primarie, esterne, vincoli UNIQUE e CHECK, fornendo evidenza dell'integrità referenziale e supportando la valutazione della normalizzazione.

5.8 X8_CHECK_DUPLICATES_AND_NULL_ORGANIZATIONS.sql

Verifica duplicati nei domini e associazioni organizzative mancanti. Il risultato ha autorizzato lo script 12, che ha bonificato tre servizi e introdotto i vincoli NOT NULL e UNIQUE.

5.9 X9_EXPORT_VIEWS_DEFINITIONS_AND_PRIVILEGES.sql

Controlla definizione, proprietà, colonne, privilegi e security_invoker delle viste. Ha evidenziato la moltiplicazione delle righe nella vista di reporting, poi risolta dallo script 18.

5.10 X10_EXPORT_RLS_POLICIES_AND_GRANTS.sql

Esporta policy RLS, ruoli e grant. Verifica l'assenza di privilegi per anon, l'assenza di DELETE per authenticated e il corretto indirizzo docentepegaso@gmail.com.

5.11 X11–X12: readiness del modello fornitori

Queste verifiche preliminari controllano lo stato dei dati e dei vincoli necessari per introdurre la gerarchia fornitori e la relazione asset–fornitore. Il loro ruolo è prevenire errori di migrazione e assicurare la compatibilità con le tabelle esistenti.

5.12 X13_CHECK_SUPPLY_CHAIN_MULTILIVELLO.sql

Censisce asset, servizi, fornitori e dipendenze, verifica orfani e distingue le relazioni dirette servizio–fornitore da quelle ricostruibili attraverso gli asset. L'esito ha autorizzato lo script 17.

5.13 X14: diagnostica della vista di reporting

Dimostra che la versione originaria di vista_reporting_servizi_critici produce una moltiplicazione cartesiana quando un servizio possiede più asset e più fornitori. La verifica guida la correzione dello script 18.

5.14 X15_CHECK_AUDIT_SYSTEM_READINESS.sql

Censisce funzioni, trigger, copertura delle tabelle, qualità dei payload, dati legacy e configurazione RLS dell'audit. Rileva la copertura limitata alla tabella asset e autorizza l'estensione generalizzata dello script 19.

5.15 X16_CHECK_ARCHIVIAZIONE_LOGICA_READINESS.sql

Individua i marker temporali esistenti, le dieci tabelle prive di archiviazione, i privilegi DELETE, la copertura audit e i vincoli CASCADE. L'esito PRONTO_PER_UNIFORMAZIONE guida lo script 20.

5.16 X17_CHECK_VERIFICA_FINALE_BACKEND.sql

Esegue sedici controlli su struttura, RLS, privilegi, DELETE, audit, archiviazione, vincoli, indici, viste, funzioni e coerenza dei dati. Dopo lo script 21 restituisce VERIFICA_FINALE_BACKEND_SUPERATA.

6. Analisi delle ridondanze e refactoring controllato

Nel contesto di uno sviluppo incrementale, la sovrapposizione temporanea di alcune istruzioni non rappresenta automaticamente un difetto. Quando è documentata e gestita da migrazioni

idempotenti, essa descrive l'evoluzione del sistema e consente di mantenere la tracciabilità delle decisioni.

6.1 Evoluzione dell'Audit Engine

La ricostruzione delle versioni precedenti documenta l'evoluzione del refactoring; lo stato operativo finale è costituito dal sistema di audit generalizzato introdotto dallo script 19.

6.2 Abilitazione RLS reiterata

L'abilitazione RLS compare in più migrazioni perché le nuove tabelle devono essere protette al momento della loro introduzione. Ripetere `ALTER TABLE ... ENABLE ROW LEVEL SECURITY` su una tabella già protetta è un'operazione sicura e idempotente. La diagnostica X17 conferma che tutte le trenta tabelle public hanno RLS attiva.

6.3 Riallineamento delle policy di lettura

Le versioni intermedie delle policy sono state sostituite tramite `DROP POLICY IF EXISTS` e ricreazione controllata. Il criterio finale è uniforme: lettura con MFA aal2 oppure utenza docente autorizzata; inserimento e aggiornamento con aal2; nessuna cancellazione applicativa.

6.4 Correzione della vista di reporting

La moltiplicazione delle righe non viene nascosta nella documentazione: è mantenuta come evidenza di un difetto diagnosticato e corretto. X14 ne dimostra la causa, mentre lo script 18 introduce aggregazioni indipendenti di asset e fornitori e X17 verifica l'assenza di duplicati.

6.5 Dalla cancellazione fisica alla conservazione logica

La baseline conteneva alcuni vincoli CASCADE, appropriati alla fase di sviluppo e seed ma non coerenti con il requisito finale di conservazione. Gli script 20 e 21 convertono tali vincoli in RESTRICT e aggiungono trigger che bloccano DELETE. La scelta privilegia auditabilità, storicità e ricostruibilità delle dipendenze.

7. Modello relazionale, gerarchie e normalizzazione

Il modello finale è composto da trenta tabelle organizzate in domini funzionali: organizzazione e responsabilità; asset e vulnerabilità; servizi e dipendenze; fornitori e Supply Chain; tassonomia ed eventi; audit e tracciabilità. Le tabelle associative separano le relazioni multi-a-molti dalle entità principali, mentre le gerarchie utilizzano tabelle autonome e metadati di validità.

Le chiavi numeriche interne garantiscono efficienza referenziale; i codici applicativi stabili, quali `codice_asset`, `codice_servizio` e `codice_fornitore`, rendono invece le entità identificabili nei flussi di integrazione e nei documenti. Questa distinzione riduce l'accoppiamento tra identificazione tecnica e identificazione di dominio.

Il modello è stato progettato secondo criteri di normalizzazione ed è sostanzialmente compatibile con la terza forma normale: gli attributi descrivono la chiave della rispettiva relazione, i dizionari sono separati dalle entità operative e le associazioni multi-a-molti sono risolte tramite tabelle specifiche. Una certificazione formale completa della 3FN richiederebbe tuttavia una matrice esplicita delle dipendenze funzionali; la relazione evita quindi di presentare come dimostrato ciò che è supportato soprattutto dalla struttura e dai vincoli.

7.1 Gerarchie storizzabili

Le gerarchie servizio–sottoservizio, asset–sotto-asset e fornitore–subfornitore adottano un modello uniforme: chiave tecnica della relazione, flag attiva, periodo di validità, relazione primaria e controlli contro cicli e autorelazioni. La storizzazione consente di chiudere una relazione senza perdere l’evidenza del legame precedente.

7.2 Integrità referenziale e domini

Chiavi esterne RESTRICT impediscono la rimozione di record ancora referenziati. Vincoli UNIQUE proteggono codici e domini; vincoli CHECK controllano stato temporale, criticità, passi del wizard e consistenza delle relazioni. X17 conferma che non esistono vincoli non validati né indici non validi.

8. Supply Chain multilivello

Il modello Supply Chain non si limita al collegamento diretto tra servizio e fornitore. La combinazione delle gerarchie e delle relazioni introdotte consente di ricostruire catene multilivello e di distinguere le dipendenze esplicite da quelle derivate.

Livello	Struttura	Finalità
Servizio → servizio	servizio_componente	Scomposizione funzionale dei servizi
Servizio → asset	servizio_dipendenza_asset	Infrastruttura necessaria all’erogazione
Asset → asset	asset_componente	Composizione tecnica e sotto-asset
Asset → fornitore	asset_fornitore	Fornitura, manutenzione, hosting o supporto
Servizio → fornitore	servizio_dipendenza_fornitore	Dipendenza contrattuale o operativa diretta
Fornitore → fornitore	fornitore_relazione	Subfornitura e catena di approvvigionamento

La vista `vista_supply_chain_multilivello` espone in modo uniforme le relazioni dirette servizio–fornitore e quelle derivate attraverso gli asset. Le tre viste gerarchiche espanse rendono consultabili i percorsi di servizio, asset e fornitore. Tutte le viste sono `security_invoker`, quindi rispettano l’identità e le policy del chiamante.

Il modello ammette che alcune dipendenze dirette non siano coperte da un asset. Questa condizione non è trattata automaticamente come errore: può rappresentare un servizio professionale, una licenza, un contratto o una dipendenza organizzativa non riconducibile a un singolo bene tecnologico.

9. Sicurezza: RLS, MFA, privilegi e viste

La sicurezza del backend è costruita su livelli complementari. I privilegi PostgreSQL limitano le operazioni disponibili ai ruoli applicativi; le policy RLS filtrano l’accesso in base al contesto

JWT; le viste `security_invoker` preservano le regole delle tabelle sottostanti; i trigger impediscono operazioni non ammesse dal modello di conservazione.

9.1 Lettura e scrittura

La lettura è consentita alle sessioni autenticate con `assurance level aal2` oppure all'utenza `docentepegaso@gmail.com`. L'inserimento e l'aggiornamento delle tabelle operative richiedono MFA `aal2`. Le tabelle di dominio e audit rimangono in sola lettura per il frontend, salvo le specifiche operazioni previste dalle rispettive migrazioni.

9.2 Ruolo anon

Il ruolo `anon` non dispone di privilegi sulle tabelle e sulle viste applicative. X17 rileva zero privilegi `anon` e zero viste accessibili `anon`. Questa scelta riduce l'esposizione della superficie dati e impedisce che una richiesta non autenticata raggiunga direttamente il modello relazionale.

9.3 Cancellazione

Non esistono policy `DELETE` né privilegi `DELETE` per `authenticated`. A questa protezione applicativa si aggiungono quattordici trigger `BEFORE DELETE`, che impediscono la cancellazione fisica anche durante operazioni amministrative ordinarie. La protezione è quindi difensiva in profondità.

9.4 FORCE ROW LEVEL SECURITY

Le tabelle utilizzano RLS senza `FORCE ROW LEVEL SECURITY`. Questa configurazione è coerente con il modello corrente, nel quale i ruoli applicativi sono soggetti alle policy mentre il proprietario del database mantiene le capacità amministrative necessarie alle migrazioni. Il comportamento è stato considerato accettabile nel perimetro del Project Work.

10. Audit e tracciabilità

Il sistema di audit consolidato è uno degli elementi centrali del backend. La funzione `fn_audit_generico`, eseguita come `SECURITY DEFINER` con `search_path` controllato, registra le operazioni `INSERT`, `UPDATE` e `DELETE` sulle quattordici tabelle operative coperte. Il frontend non scrive direttamente in `audit_log`.

Dimensione	Informazioni registrate
Contesto dell'oggetto	Tabella, tipo di entità, identificativo, chiave, codice e nome
Contesto funzionale	Riferimenti ad asset, servizio e fornitore quando disponibili
Identità dell'attore	UUID, e-mail, AAL, ruolo JWT, ruolo database e origine
Operazione	<code>INSERT</code> , <code>UPDATE</code> o <code>DELETE</code> con data e ora
Payload	Valore precedente e nuovo in formato JSONB
Consultazione	<code>vista_audit_dettagliato</code> con contesto leggibile e <code>security_invoker</code>

Gli eventi legacy prodotti dalle versioni precedenti non vengono eliminati: lo script 19 li normalizza e conserva. La verifica funzionale con sessione JWT aal2 ha confermato la registrazione dell'e-mail, dell'UUID e del contesto authenticated. X17 verifica inoltre la corrispondenza numerica tra audit_log e vista_audit_dettagliato.

La presenza del trigger DELETE nell'audit non autorizza la cancellazione fisica. Dopo lo script 20, il trigger di blocco interviene prima che l'operazione venga completata. L'audit rimane comunque predisposto a documentare l'intero spettro DML e conserva la compatibilità con le fasi precedenti della pipeline.

11. Archiviazione logica e conservazione storica

Il requisito finale prevede che le entità operative non vengano eliminate fisicamente. Lo script 20 uniforma il modello distinguendo due famiglie di tabelle.

Famiglia	Copertura	Metadati
Modello standard	10 tabelle	attiva, archiviato_il, archiviato_da, motivo_archiviazione
Modello temporale	4 tabelle	attiva, valido_dal, valido_al

Nelle tabelle standard, la funzione fn_gestisci_archiviazione_logica valorizza automaticamente data e utente quando attiva passa a false e rimuove i metadati quando il record torna attivo. Le relazioni storicizzate mantengono invece il proprio periodo di validità e i vincoli già introdotti dagli script 13–16.

La funzione fn_blocca_cancellazione_fisica viene richiamata da quattordici trigger BEFORE DELETE. In parallelo, tutti i vincoli ON DELETE CASCADE sono stati sostituiti con RESTRICT. L'ultimo vincolo residuo, tra responsabile_ruolo e ruolo, è stato individuato da X17 e corretto dallo script 21.

X17 conferma zero anomalie di coerenza: nessun record attivo possiede una data di archiviazione e nessun record inattivo ne è privo; nelle tabelle temporali, valido_al è coerente con attiva e non precede valido_dal.

12. Reporting e interoperabilità ACN/NIS2

Le viste separano il modello fisico dalle esigenze di consultazione, esportazione e integrazione. La configurazione security_invoker garantisce che la vista non aggiri la RLS delle tabelle sottostanti.

Vista	Scopo
vista_esportazione_acn_assets	Esportazione strutturata di asset, categorie e vulnerabilità
vista_reporting_servizi_critici	Una riga per servizio, asset e fornitori aggregati separatamente

Vista	Scopo
vista_gerarchia_servizi_espansa	Percorsi della gerarchia servizio-sottoservizio
vista_gerarchia_asset_espansa	Percorsi della gerarchia asset-sotto-asset
vista_gerarchia_fornitori_espansa	Percorsi della gerarchia fornitore-subfornitore
vista_supply_chain_multilivello	Dipendenze dirette e derivate della catena di fornitura
vista_audit_dettagliato	Consultazione leggibile e filtrabile degli eventi di audit

La correzione della vista di reporting costituisce un esempio di validazione guidata dai dati: il problema delle join multi-a-molti è stato prima osservato da X9, poi isolato da X14, corretto dallo script 18 e infine verificato da X17. Il risultato finale contiene quattro righe per quattro servizi distinti, senza duplicazioni.

13. Verifica finale del backend

X17 rappresenta il controllo di chiusura del backend. La prima esecuzione ha individuato un singolo vincolo ON DELETE CASCADE residuo; dopo la migrazione 21, la nuova esecuzione ha restituito esito positivo in tutte le sezioni operative.

Sezione	Controllo	Esito
01	Struttura generale	OK
02	RLS su tutte le tabelle public	OK
03	RLS sulle tabelle operative	OK
04	Privilegi anon	OK
05	Protezione DELETE	OK
06	Copertura audit	OK
07	Archiviazione logica	OK
08	Coerenza dei dati archiviati	OK
09	Vincoli e indici	OK
10	Sicurezza delle viste	OK
11	Coerenza funzionale delle viste	OK

Sezione	Controllo	Esito
12	Sicurezza delle funzioni principali	OK
13	Protezione di audit_log	OK
14	Riepilogo policy RLS	Informativo
15	Riproducibilità logica del database	OK
16	Esito finale	VERIFICA FINALE BACKEND SUPERATA

Tra le metriche finali figurano trenta tabelle, zero tabelle senza RLS, zero privilegi anon, zero policy DELETE, quattordici trigger di audit, quattordici trigger di blocco DELETE, dieci trigger di archiviazione, zero chiavi esterne CASCADE, zero vincoli non validati, zero indici non validi, sette viste presenti e nessuna anomalia di archiviazione.

La verifica del database non può certificare la presenza fisica e l'ordine dei file nel repository o nella copia locale. Questa parte della riproducibilità viene controllata attraverso la matrice documentale e il workflow di versionamento descritto nella sezione successiva.

14. Riproducibilità, repository e gestione delle versioni

La riproducibilità del progetto dipende dall'allineamento tra stato del database, file SQL e documentazione. Gli script produttivi sono numerati da 01 a 21; gli strumenti diagnostici da X1 a X17. Ogni migrazione è conservata dopo la verifica su Supabase e accompagnata da un messaggio di commit descrittivo.

Il repository è organizzato in tre aree principali: sql per migrazioni e diagnostiche, src per il codice applicativo e docs per diagrammi e relazioni tecniche. index.html e README.md si trovano nella radice. Il file docs/README.md svolge la funzione di indice documentale locale.

La pipeline non deve essere interpretata come un singolo script monolitico. Il suo valore risiede nella sequenza: ogni file documenta una decisione, una correzione o un hardening e può essere collegato alla diagnostica che lo ha preceduto o seguito. Questa tracciabilità è particolarmente importante per audit, sicurezza e modifiche dei vincoli referenziali.

La versione definitiva del documento principale del PW dovrà sintetizzare i risultati qui esposti e indicare questa relazione come allegato tecnico di approfondimento. La relazione tecnica frontend, denominata docs/Relazione_Tecnica_Frontend_Applicativo.pdf, costituirà l'allegato di approfondimento dedicato al codice applicativo e alle SCR.

15. Conclusioni

Il backend del Project Work ha completato i sette obiettivi di consolidamento definiti durante lo sviluppo: gerarchia fornitori, relazione asset-fornitore, Supply Chain multilivello, correzione del

reporting, audit generalizzato, archiviazione logica e verifica finale. Il database è completo rispetto al perimetro stabilito per l'elaborato, pur rimanendo naturalmente estendibile in presenza di nuovi requisiti.

La crescita a trenta tabelle non costituisce da sola un indicatore di qualità, ma nel progetto corrisponde alla separazione esplicita di domini, relazioni e gerarchie. La solidità deriva soprattutto dai vincoli, dalla RLS, dalla tracciabilità, dalla conservazione logica e dalla capacità di verificare lo stato finale mediante X17.

La fase successiva riguarda l'aggiornamento coordinato del modello ER, dei diagrammi del repository, di README.md, di docs/README.md e del documento principale del PW. Terminata la documentazione backend, il lavoro potrà concentrarsi sulle SCR e sull'allineamento del frontend al modello consolidato, con successiva redazione della Relazione_Tecnica_Frontend_Applicativo.pdf.

Appendice A — Matrice di controllo documentale e applicativa

Elemento	Collocazione	Stato	Ruolo
Pipeline SQL 01–21	sql/	Completata e verificata	Fonte delle modifiche produttive
Diagnostiche X1–X17	sql/	Completate e verificate	Evidenze di readiness e chiusura
Modello ER	docs/	Da aggiornare	Deve rappresentare le 30 tabelle finali
Relazione backend	docs/Relazione_Tecnica_Backend_Database.pdf	In aggiornamento	Allegato tecnico del PW
Relazione frontend	docs/Relazione_Tecnica_Frontend_Applicativo.pdf	Da creare dopo il consolidamento	Allegato tecnico del frontend
Documento principale PW	Template ufficiale	Bozza da riallineare	Documento principale di presentazione
README principale	README.md	Da aggiornare	Sintesi, struttura e accesso al progetto
Indice documentale	docs/README.md	Da aggiornare	Mappa dei documenti in docs
Codice applicativo	index.html e src/	Da consolidare	Interfaccia, logica client e integrazione Supabase
Applicativo web e SCR	GitHub Pages	Fase successiva	Test funzionali e correzioni frontend
Evidenze finali	X17 e test frontend	Backend completato	Base della revisione conclusiva

La matrice deve essere mantenuta durante le fasi successive. Ogni aggiornamento del documento principale o dell'applicativo dovrà essere confrontato con gli allegati tecnici, evitando divergenze tra ciò che viene descritto e ciò che è effettivamente implementato.